

Algoritmi Avansați 2026 — Partea I

FMI, UniBuc, Anul II

Nume, Prenume:

Grupa:

Punctaj total: 30 de puncte

Instrucțiuni: Examen cu cursurile pe masă. Grilele (1-8) au un singur răspuns corect. Întrebările deschise (9-12) cer justificare concisă. **Nu aveți voie să completați altceva decât numele pe această foaie.**

1. **(2p)** De ce abordarea de tip *Greedy* (alegerea obiectelor descrescător după raportul valoare/greutate) găsește soluția optimă pentru Rucsacul Fraționar, dar eșuează în a garanta optimul pentru Rucsacul 0/1?

- (a) Deoarece sortarea durează $\mathcal{O}(n \log n)$ și depășește timpul admis pentru 0/1.
- (b) Pentru că obiectele cu raport mic au întotdeauna valori absolute mai mari.
- (c) Pentru că problema 0/1 nu este $\mathcal{NP}$ -completă.
- (d) Deoarece, la varianta 0/1, capacitatea rămasă la final nu poate fi completată perfect, ducând la irosirea spațiului valoros din rucsac.

2. **(2p)** În algoritmul **Greedy-LB** pentru echilibrarea sarcinilor (pe m mașini), care dintre următoarele configurații de joburi reprezintă scenariul cel mai defavorabil (*worst-case*), care forțează algoritmul să obțină o aproximare slabă?

- (a) m joburi de durată uriașă, urmate de m joburi de durată infimă.
- (b) O secvență de joburi de durată foarte mică, urmată la final de un singur job de durată extrem de mare.
- (c) Toate joburile au exact aceeași durată p .
- (d) Joburi sortate strict descrescător (de la cel mai mare la cel mai mic).

3. **(2p)** Se dă un graf stea S_n (un nod central legat de $n - 1$ noduri frunză independente). Dacă aplicăm algoritmul de 2-aproximare pentru **Vertex Cover** bazat pe **Cuplaj Maximal** (Maximal Matching), ce dimensiune va avea acoperirea returnată de algoritm (ALG) față de dimensiunea optimă (OPT)?

- (a) $ALG = n$, în timp ce $OPT = 1$.
- (b) $ALG = n - 1$, în timp ce $OPT = n/2$.
- (c) $ALG = 2$, în timp ce $OPT = 1$.
- (d) $ALG = 1$, în timp ce $OPT = 1$ (algoritmul este optim pe grafuri stea).

4. (2p) Într-un **Algorithm Genetic** care utilizează o reprezentare binară a cromozomilor, ce efect va avea setarea probabilității de mutație la extremă: $p_m = 1.0$ (100%)?

- (a) Va garanta găsirea optimului global în prima generație.
- (b) Indivizii vor rămâne complet neschimbați de-a lungul generațiilor.
- (c) Fiecare cromozom se va transforma în complementul său bit cu bit la fiecare generație, distrugând complet orice schemă stabilă (structuri genetice bune).
- (d) Operatorul de încrucișare (crossover) va deveni determinist.

5. (2p) Rulăm algoritmul lui **Freivalds** pentru a testa identitatea de matrice $A \cdot B = C$. Știind că la o singură rulare probabilitatea de eroare (fals-pozitiv) este $\leq 1/2$, care este numărul **minim** de rulări independente k necesare pentru ca probabilitatea de eroare să scadă strict sub 1%?

- (a) $k = 5$
- (b) $k = 7$
- (c) $k = 10$
- (d) $k = 100$

6. (2p) Spre deosebire de Quicksort-ul clasic, varianta **Randomized Quicksort** alege pivotul aleatoriu uniform. Care este complexitatea *worst-case* absolută (nu cea așteptată/expected) a acestui algoritm randomizat pe un vector de n elemente?

- (a) $\mathcal{O}(n \log n)$
- (b) $\mathcal{O}(n \log^2 n)$
- (c) $\mathcal{O}(n^2)$
- (d) $\mathcal{O}(n)$

7. (2p) Într-o **Skip List**, modificăm moneda ideală (probabilitate $p = 1/2$) cu una care măsluiește promovarea la nivelul superior cu probabilitatea $p = 1/4$. Care este consecința directă asupra structurii?

- (a) Scade numărul așteptat de pointeri în memorie, dar crește lungimea așteptată a căutării orizontale pe fiecare nivel.
- (b) Crește înălțimea totală a structurii, devenind un arbore binar de căutare complet.
- (c) Toate elementele vor ajunge obligatoriu pe nivelul maxim.
- (d) Căutarea devine $\mathcal{O}(1)$ în medie.

8. **(2p)** De ce **nu** se pot șterge elemente dintr-un **Bloom Filter** clasic prin simpla resetare a celor k biți la valoarea 0?

- (a) Deoarece funcțiile hash nu sunt inversabile, deci nu știm ce biți să ștergem.
- (b) Deoarece acei biți ar putea fi setați pe 1 și de alte elemente prezente; resetarea lor ar genera erori de tip „fals-negativ” pentru acele elemente.
- (c) Deoarece structura ar trebui re-alocată complet în memorie la fiecare ștergere.
- (d) Ștergerea este posibilă în varianta clasică, dar durează $\mathcal{O}(m)$.

9. **(3.5p)** *Algoritmi Genetici (Convergență Prematură)*: În contextul selecției tip Ruletă (*Fitness Proportionate Selection*), explicați mecanismul prin care apariția timpurie a unui singur individ excepțional de bun („Super-Individ”) blochează explorarea și duce algoritmul într-un optim local.

10. **(3.5p)** *TSP General vs. Inegalitatea Triunghiului*: Algoritmul Double-Tree transformă un circuit eulerian pe un arbore de acoperire minim într-un tur TSP valid utilizând procedeul de „scurtcircuitare” (shortcut) a nodurilor deja vizitate. Explicați precis de ce această scurtcircuitare poate **crește** enorm costul turului dacă se renunță la inegalitatea triunghiului.

11. **(3.5p)** *Load Balancing (LPT)*: Considerăm o instanță cu $m = 3$ mașini. Oferiți un exemplu cu un număr mic de joburi (precizând duratele lor) unde algoritmul **LPT** (Longest Processing Time) nu reușește să găsească soluția optimă, deși este mai performant decât varianta Greedy simplă.

12. **(3.5p)** *Amplificarea Succesului (Monte Carlo)*: Dispuneți de un algoritm Monte Carlo A care rezolvă o problemă de decizie. Dacă răspunsul corect este DA, returnează DA întotdeauna. Dacă răspunsul corect este NU, se înșală și returnează DA cu probabilitatea $1/3$. Construiți un algoritm B care folosește A ca "black-box" și scade probabilitatea de eroare pe instanțele NU la sub 1.5%. Câte apeluri ale algoritmului A va face algoritmul B ? Justificați matematic.

BAREM ȘI REZOLVĂRI BATERIA NOUĂ

Răspunsuri Grilă (2p fiecare)

1. Deoarece, la varianta 0/1, capacitatea rămasă la final nu poate fi completată perfect, ducând la irosirea spațiului valoros din rucsac.
2. O secvență de joburi de durată foarte mică, urmată la final de un singur job de durată extrem de mare.
3. $ALG = 2$, în timp ce $OPT = 1$.
4. Fiecare cromozom se va transforma în complementul său bit cu bit la fiecare generație, distrugând complet orice schemă stabilă (structuri genetice bune).
5. $k = 7$ (explicație: $1/2^k < 0.01 \Rightarrow 2^k > 100 \Rightarrow k \geq 7$)
6. $\mathcal{O}(n^2)$
7. Scade numărul așteptat de pointeri în memorie, dar crește lungimea așteptată a căutării orizontale pe fiecare nivel.
8. Deoarece acei biți ar putea fi setați pe 1 și de alte elemente prezente; resetarea lor ar genera erori de tip „fals-negativ” pentru acele elemente.

Explicații Întrebări Deschise (3.5p fiecare)

9. Super-Individul în AG (C5)

Barem: Un "Super-Individ" are o valoare de fitness disproporționat de mare comparativ cu restul populației curente. Într-o selecție pe bază de ruletă, felia sa din "roată" va fi uriașă (**1.5p**). Acest lucru face ca el să fie selectat de repetate ori pentru încrucișare, umplând generația următoare cu clonele/copiii săi (**1p**). Drept urmare, diversitatea genetică a populației se prăbușește brusc, iar algoritmul converge prematur către regiunea din spațiul de căutare a acelui individ (care poate fi doar un optim local), ignorând explorarea altor zone promițătoare (**1p**).

10. Scurtcircuitarea în lipsa Inegalității Triunghiului (C3)

Barem: Scurtcircuitarea elimină vizitarea unui nod intermediar (ex: merge direct de la A la C , sărind peste B). În Metric TSP, inegalitatea triunghiului ($dist(A, C) \leq dist(A, B) + dist(B, C)$) garantează că această muchie nouă, directă, nu poate fi mai costisitoare decât calea originală (**1.5p**). Însă, în General TSP, unde ponderile pot fi complet arbitrare, costul direct $dist(A, C)$ poate fi un număr colosal (ex: ∞), deci aplicarea oricărui shortcut pe circuitul eulerian poate exploda necontrolat costul turului, anulând orice garanție de aproximare (algoritmul putând genera un rezultat

arbitrar de rău) **(2p)**.

11. Contraexemplu LPT (C2)

Barem: Punctajul maxim se acordă pentru orice contraexemplu valid care demonstrează că LPT nu obține optimul **(1.5p)** + demonstrația distribuirii pe mașini **(2p)**.

12. Construirea lui B - Monte Carlo (C6)

Barem: Algoritmul B va rula algoritmul A de k ori în mod independent pe aceeași instanță. Dacă A returnează de fiecare dată DA, B returnează DA. Dacă A returnează cel puțin un NU, B va returna ferm NU (deoarece eroarea este unilaterală și un singur NU garantează că instanța e negativă) **(1.5p)**. Probabilitatea ca B să se înșele (să returneze DA pe o instanță NU) se întâmplă doar dacă A se înșală de k ori la rând: $P_{\text{eroare}} = (1/3)^k$ **(1p)**. Vrem ca $(1/3)^k < 0.015 \Leftrightarrow 1/3^k < 3/200 \Leftrightarrow 3^k > 66.6$. Știm că $3^3 = 27$ și $3^4 = 81$. Deci este nevoie de $k = 4$ apeluri independente **(1p)**.